BIG DATA ANALYTICS BASICS USING PYSPARK

TECHNICAL INTERNSHIP GRADUATION REPORT

Prepared By: Arshpreet Singh

Internship: Data Analytics Internship

Organization: Edutech Solution

Submission Date: July 2, 2026

Status: Final Evaluation Copy

2. Acknowledgement

I would like to express my deep sense of gratitude and respect to **Edutech Solution** for providing me with the wonderful opportunity to undergo this Data Analytics Internship. This technical report marks the culmination of an intensive hands-on experience focused on practical large-scale data processing and distributed computing environments.

I am profoundly grateful to my industry mentors and technical supervisors who guided me throughout the project implementation phases. Their constructive criticism, expert architectural feedback, and rigorous evaluation methodologies helped me bridge the gap between academic theory and industry-scale data engineering workflows. The opportunity to work with real-world infrastructure logs via advanced frameworks like Apache Spark has significantly enhanced my capability to manipulate, clean, and extract hidden insights from complex high-volume datasets.

Lastly, I wish to extend my appreciation to all my peer interns and the entire learning community at Edutech Solution for fostering a collaborative and highly intellectual technical environment that inspired continuous learning and continuous validation.

3. Table of Contents

Section Number & Title	Focus Area
1. Cover Page	Administrative Framework & Title Details
2. Acknowledgement	Corporate & Academic Credits
3. Table of Contents	Document Schema and Navigation Map
4. Executive Summary	Project Overview, Tech Stack, and Deliverables
5. Introduction	Contextualizing Big Data and Apache Spark Ecosystem
6. Objectives of the Project	Core Technical Goals & Strategic Milestones
7. Big Data Fundamentals	The 5 Vs Model & Paradigms of Scale
8. Hadoop Overview	Architectural Mapping: HDFS, YARN, MapReduce
9. Apache Spark & PySpark	In-Memory Computing, RDDs, and DataFrames
10. MapReduce Concept	Functional Paradigms: Map, Shuffle, and Reduce
11. Dataset Description	Deep Dive into client_hostname.csv Structure
12. Methodology	Step-by-Step Computational Workflow
13. Implementation	Annotated Source Code and Execution Strategies
14. Results and Analysis	Empirical Observations and Data Metric Breakdown
15. Key Findings	Top 10 Architectural and Operational Insights
16. Challenges Faced	Technical Bottlenecks, Constraints, and Resolution Strategy
17. Learning Outcomes	Competency Framework & Hard/Soft Skills Acquired
18. Future Scope	Real-Time Streams, ML Pipelines, and Cloud Scaling
19. Conclusion	Final Technical Summary and Impact Valuation
20. References	Academic Papers, Formal Documentations & Manuals

4. Executive Summary

This technical report provides a comprehensive architectural documentation of the project titled **"Big Data Analytics Basics Using PySpark"** executed during the Data Analytics Internship at **Edutech Solution**. As enterprise networks grow exponentially, analyzing massive infrastructure access records becomes a critical operational requirement. This project addresses the ingestion, parsing, quality audit, and exploratory analysis of a network tracking dataset containing 258,445 connection records across distributed environments.

The core framework utilized is **Apache Spark** accessed via its native Python API, **PySpark**, deployed inside an interactive **Google Colab** computing instance. The underlying dataset, `client_hostname.csv`, captures multidimensional enterprise logging features including network clients, hostnames, nested string alias matrices, and IP destination arrays. A key challenge successfully addressed during implementation was handling the high volume of incomplete or failed lookups, as evidenced by a substantial portion (97.7%) of records displaying explicit socket exceptions and structural indicators like `[Errno 1] Unknown host` or literal `NULL` references.

Through systematic data engineering operations—including SparkSession bootstrap, automated schema inference, programmatic null filtering, custom User Defined Functions (UDFs) utilizing Python's Abstract Syntax Tree (`ast`) parsing, and distributed aggregations—the raw dataset was parsed into clean relational models. Key outcomes include isolating 5,819 completely resolved high-value network transactions, extracting 5,883 unique historical network aliases, and classifying structural endpoint typologies. Ultimately, this internship project demonstrates how PySpark handles distributed high-cardinality records gracefully where traditional relational engines or single-threaded pandas workflows fail due to memory constraints or lack of distributed computation primitives.

5. Introduction

What is Big Data?

In the contemporary digital era, Big Data refers to datasets whose absolute structural sizes, computational complexities, and generation frequencies surpass the processing thresholds of conventional single-node database management applications. Rather than simply indicating massive volumetric scales, Big Data represents an architectural state shift where data can no longer be modeled via simple tabular relationships or processed inside a single computer's random-access memory (RAM).

Importance of Big Data

The strategic deployment of Big Data paradigms allows modern digital enterprises to capture ambient telemetry that was historically discarded due to storage constraints. Continuous auditing of clickstreams, global server access logs, real-time sensory inputs, and transactional matrices provides organizations with predictive foresight, optimized customer experiences, and immediate network threat mitigation vectors.

Real-World Applications

Big Data applications span diverse global domains. In financial engineering, high-frequency fraud detection loops analyze millisecond-level credit patterns. In healthcare informatics, parallel sequence processors correlate millions of genetic configurations to synthesize tailored medications. Similarly, telecom conglomerates run continuous analytics over network routing matrices to avoid hardware congestion and route packet transfers dynamic-optimally.

The Need for Distributed Computing

According to physical hardware scaling constraints, single-core processor clock frequencies hit structural thresholds due to thermal dissemination boundaries. To scale compute capacity linearly, the data engineering discipline shifted away from horizontal machine scale-up (buying a more expensive single computer) toward vertical cluster scale-out (combining hundreds of low-cost commodity servers together). Distributed computing networks split massive data files across a coordinated mesh of computing nodes, performing localized analytics concurrently and combining intermediate results over optimized internal networking layers.

Role of Apache Spark

Apache Spark emerged as the dominant unified open-source engine for distributed large-scale data manipulation. Unlike its predecessor frameworks that relied heavily on disk-bound batch computations, Spark introduces an optimized cluster compute paradigm utilizing state-of-the-art memory allocation layouts, allowing fast iterative pipeline execution across heterogeneous computing clusters.

6. Objectives of the Project

The principal objectives driving the design and execution of this data engineering project are defined as follows:

- **Formulate Foundational Big Data Literacy:** Deeply analyze the architectural mechanisms governing large-scale distributed setups, establishing a professional conceptual framework around cluster topologies, cluster resource schedulers, and distributed file structures.
- **Master the PySpark API Interface:** Acquire hands-on professional competence in writing robust Python code interacting with the Apache Spark core, utilizing structural abstractions like SparkSessions, DataFrames, Spark SQL functions, and distributed execution graphs.
- **Implement Distributed Data Preprocessing Pipelines:** Develop scalable programmatic workflows to identify, quantify, filter, and normalize massive datasets containing sparse features, invalid characters, nested arrays, and complex system exceptions.
- **Execute Exploratory Data Analysis (EDA) at Scale:** Apply distributed grouping, multi-stage sorting, categorical aggregations, and pattern filtering over high-volume database collections without exceeding target container memory boundaries.

- **Acquire Professional System Engineering Insights:** Build practical skills in deploying, monitoring, and cleanly terminating ephemeral cloud compute environments like Google Colab while tracking resource utilisation profiles.

7. Big Data Fundamentals

Understanding the modern data engineering landscape requires a rigorous contrast between legacy transactional architectures and distributed big data environments.

Traditional Data vs. Big Data

Traditional data setups rely heavily on centralized Relational Database Management Systems (RDBMS) governed by strict ACID (Atomicity, Consistency, Isolation, Durability) guarantees, handling structured entries written in predefined schemas. Conversely, Big Data architectures handle highly amorphous, semi-structured, or unstructured text payloads across distributed filesystems with dynamic, late-binding schema application models.

The 5 Vs Model of Big Data

The foundational characteristics that separate Big Data configurations are formalised via the 5 Vs framework:

- **Volume:** The scale of data injected into the system, measuring from Terabytes up to Exabytes. *Example:* Global social networks ingesting petabytes of user telemetry daily.
- **Velocity:** The rate at which incoming data streams flow into processing engines. *Example:* Financial trading engines processing millions of sensor notifications per second.
- **Variety:** The structural diversity of input data formats, including JSON blocks, raw server logs, multimedia binary objects, and audio streams. *Example:* E-commerce platforms combining relational tables with text reviews and image vectors.
- **Veracity:** The inherent cleanliness, reliability, and truthfulness of the target dataset. *Example:* Network audit streams containing fragmented records, dropped packages, and missing fields that require algorithmic filtering.
- **Value:** The tangible operational insight or business intelligence derived from applying processing algorithms to raw records. *Example:* Predicting server downtime based on historic trends in access exception codes.

8. Hadoop Overview

Apache Hadoop represents a foundational open-source framework designed to store and process large distributed datasets across commodity hardware clusters using basic programming models.

Hadoop Core Architecture

The classical Hadoop architecture consists of three interconnected layers:

- **Hadoop Distributed File System (HDFS):** A block-oriented distributed file layout designed to run on commodity physical nodes. It breaks big files into blocks (typically 128MB) and distributes them across multiple machines, maintaining high fault-tolerance by replicating each chunk across distinct racks.
- **YARN (Yet Another Resource Negotiator):** The centralized cluster manager responsible for arbitrating hardware resources (CPU, RAM, disk) across competing applications. It decouples resource monitoring from data processing models.
- **MapReduce:** The native software processing paradigm that implements parallelized, disk-backed batch algorithms over HDFS blocks.

Advantages and Limitations of Hadoop

While Hadoop provided low-cost storage scaling and exceptional fault tolerance, its MapReduce processing model suffered from severe operational latencies. Because MapReduce enforces strict disk serialization between the map and reduce execution boundaries, iterative applications (such as machine learning loops) spend major compute windows performing slow disk I/O operations, limiting real-time interaction capabilities.

9. Apache Spark & PySpark

Apache Spark was designed directly to overcome the operational limitations of Hadoop MapReduce by executing computations entirely within the cluster's Random Access Memory (RAM).

Spark Architectural Paradigm

Spark operates on a master-slave topology managed via a centralized Driver program that coordinates with an active Cluster Manager (such as YARN, Mesos, or Spark's Standalone Scheduler) to execute tasks over independent worker nodes called Executors.

[Image of Apache Spark Cluster Architecture: Driver Program, Cluster Manager, and Worker Nodes with Executors]

Figure 1: High-Level Apache Spark Distributed Execution Topology

Core Abstractions: RDDs vs. DataFrames

The foundation of Spark data manipulation was built on **Resilient Distributed Datasets (RDDs)**—fault-tolerant collections of objects partitioned across cluster machines that can be updated via parallel transformations. Modern Spark implementations use **Spark DataFrames**, which wrap high-level structural semantics around RDDs, formatting data into named columns. DataFrames leverage the internal **Catalyst Optimizer** to dynamically compile query statements into highly optimized distributed physical execution plans.

PySpark and Its Core Advantages

PySpark is the official Python language binding for Apache Spark, blending the expressiveness and rich data science ecosystem of Python with the immense distributed computation power of the Spark engine. PySpark's major advantages include in-memory processing speeds up to 100x faster than traditional MapReduce, lazy evaluation workflows that defer data movement until final action triggers, and seamless cross-compatibility with deep storage structures like HDFS, Amazon S3, and local directories.

10. MapReduce Concept

Despite Spark's reliance on in-memory execution, it still leverages the functional fundamentals of the MapReduce data routing pattern during broad cluster-wide reshuffling steps.

The Three Functional Phases

1. **Map Phase:** Distributed input nodes process raw input blocks sequentially, transforming single lines into intermediate key-value pairs without side effects.
2. **Shuffle & Sort Phase:** The system automatically redistributes intermediate outputs across the cluster network, grouping all records sharing identical keys onto the same target reducer machine.
3. **Reduce Phase:** The target execution nodes consume the grouped key-value arrays concurrently, applying user-defined aggregation code to combine values into a single compressed output dataset.

Word Count Conceptual Mechanics

Consider a simple text input processed via MapReduce: `"Spark Spark MapReduce"`. The workflow maps this line into intermediate pairs: `("Spark", 1), ("Spark", 1), ("MapReduce", 1)`. The shuffle phase aggregates these pairs across network channels into discrete keys: `"Spark" → [1, 1]; "MapReduce" → [1]`. The reduce phase sums the collection array, writing out final consolidated statistics: `("Spark", 2), ("MapReduce", 1)`.

11. Dataset Description

The project uses a structured networking dataset titled `client_hostname.csv`. It contains transaction logs mapping access endpoints to host parameters. Below is an architectural overview of its attributes:

Column Identifier	Inferred Type	Nullability	Functional Description
`client`	String	True	The origin IPv4 network address initiating the transactional access request.
`hostname`	String	True	The resolved network host target or domain identifier associated with the client.
`alias_list`	String	True	A string-serialized Python list containing domain aliases or socket error traces.
`address_list`	String	True	A string-serialized array mapping resolved alternative IP addresses for the target host.

[Insert Figure 2: Dataset Preliminary Ingestion Preview Screenshot from PySpark DataFrame]

12. Methodology

The engineering methodology applied to execute this project is designed around a structured, sequential workflow optimized for distributed datasets:

[Insert Workflow Diagram: Environment Setup → Ingestion → Data Quality Audit → UDF Parsing → Categorical Aggregation → Resource Release]
Figure 3: Distributed Data Processing Life-Cycle Methodology

The processing architecture unfolds as follows: First, the virtual environment is prepared by loading external Java dependencies and compiling PySpark libraries. Second, an explicit `SparkSession` entry point is constructed to instantiate driver-executor configurations. Third, raw CSV records are read with late-binding automated data type inference models. Fourth, a structural audit scans columns to map data sparsity, tracking both literal null references and string-based exception payloads. Fifth, custom Python Abstract Syntax Tree algorithms are applied over arrays via Spark User Defined Functions (UDFs). Sixth, high-cardinality GroupBy filters isolate active endpoints, and lastly, resources are cleanly unmounted to avoid execution leaks.

13. Implementation

This section provides a rigorous breakdown of every functional block executed during the computational process, mapping code implementations to their distributed objectives.

Step 1 & 2: Environment Initialization & Library Import

Objective: Provision the virtual operating system container with the necessary Apache Spark binaries and pull the relational entry points into the Python execution space.

```
# Install PySpark engine via pip package manager
!pip install pyspark

# Import structural SparkSession entry point from SQL library
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum, count, countDistinct, desc, asc, when, explode,
udf
from pyspark.sql.types import ArrayType, StringType
import ast
```

Step 3: SparkSession Construction

Objective: Instantiate the logical driver context and register the application signature within the active cluster workspace.

```
# Construct Spark session utilizing builder pattern config
spark = SparkSession.builder     .appName("BigDataAnalyticsBasics")     .getOrCreate()

print("SparkSession successfully instantiated.")
```

Step 5: Data Ingestion and Structural Parsing

Objective: Ingest the raw data file into a distributed DataFrame while dynamically reading header metadata.

```
# Define absolute file locator path
dataset_path = "/content/client_hostname.csv"

# Ingest data with automated inferSchema flags
df = spark.read.csv(dataset_path, header=True, inferSchema=True)
```

Step 11: Programmatic Multi-Typology Missing Value Analysis

Objective: Audit data completeness by tracking both standard system nulls and literal 'NULL' text placeholders generated by legacy loggers.

```
missing_values_counts = []

for column in df.columns:
    # Query count of native null records
    null_count = df.filter(col(column).isNull()).count()

    # Check for string literal 'NULL' overrides in character columns
    if dict(df.dtypes)[column] == 'string':
        null_string_count = df.filter(col(column) == 'NULL').count()
    else:
        null_string_count = 0

    total_missing = null_count + null_string_count
    if total_missing > 0:
        missing_values_counts.append((column, total_missing))
```

Step 17: Filter Valid Network Transactions

Objective: Cleanly separate successfully resolved network requests from socket exception failures using structural filters.

```
# Filter rows matching strict data integrity and network resolution parameters
filtered_df = df.filter(
    (~col("alias_list").contains("[Errno 1] Unknown")) &
    (col("address_list").isNotNull()) &
    (col("address_list") != 'NULL')
)
```

Step 19.1: Abstract Syntax Tree (AST) UDF Array Extraction

Objective: Parse string-serialized Python lists into native Spark Array types to support relational explosion operations.

```
def parse_alias_list(alias_str):
    if alias_str and alias_str.startswith('[') and alias_str.endswith(']'):
        try:
            parsed_list = ast.literal_eval(alias_str)
            if isinstance(parsed_list, list):
                return parsed_list
        except (ValueError, SyntaxError):
            pass
    return []

# Bind Python literal compiler into distributed execution space
parse_alias_list_udf = udf(parse_alias_list, ArrayType(StringType()))

# Process column transformations and explode nested arrays into distinct rows
parsed_aliases_df = filtered_df.withColumn("parsed_aliases",
parse_alias_list_udf(col("alias_list")))
individual_aliases_df = parsed_aliases_df.withColumn("alias",
explode(col("parsed_aliases")))
```

Step 20: Resource Release and Driver Deconstruction

Objective: Explicitly close down the Spark session to release memory allocations and prevent resource leaks.

```
# Deconstruct context
spark.stop()
print("SparkSession gracefully terminated.")
```

14. Results and Analysis

Executing the data engineering pipeline yielded highly specific metric distributions across the target computing workspace:

DataFrame Scale and Core Characteristics

The initial data ingestion phase confirmed that the `client_hostname.csv` dataset contains a total of **258,445 records** across **4 attributes**. Automated data type checking classified all incoming columns (`client`, `hostname`, `alias_list`, `address_list`) as structural `StringType` attributes.

[Insert Figure 4: Schema Composition Output Diagram - printSchema Terminal Trace]

Data Quality Audit & Completeness Matrix

The multi-typology missing value check revealed extensive data sparsity. The `client` column showed zero missing entries. However, the `hostname` feature had 4 missing records. The `address_list` column presented an exceptionally high missing count of **252,626 records**, indicating that approximately 97.74% of the captured records failed to resolve to an alternative destination IP address during log tracking.

Exploratory Data Analysis Metric Matrix

A count-distinct check over the raw parameters demonstrated that the dataset contains **258,445 unique clients**, indicating that every individual row originates from a distinct source IP block. Conversely, the dataset contains **258,273 unique hostnames**, showing a small volume of recurring targets across different source nodes.

Target Hostname Identifier	Aggregated Occurrences	Percentage Share (%)
`int0.client.access.fanaptelecom.net`	101	0.0390%
`unknown.puregig.net`	31	0.0120%
`server2us.getmyip.co`	5	0.0019%
`swe-net-ip.as51430.net`	5	0.0019%
`NULL` (Literal text fallback)	4	0.0015%
`zmap.sorengard.com`	4	0.0015%
`host.coloup.com`	4	0.0015%

Advanced UDF Array Extraction Insights

Filtering out failed transactions isolated exactly **5,819 highly reliable, fully resolved records**. Applying the Abstract Syntax Tree array parser across these records extracted a total of **5,883 unique network aliases** from the `alias_list` strings. Cross-referencing clients through an intersection join between successful and failed records returned a count of **0**, proving that clients in this log archive exhibit completely mutually exclusive behaviors; a source IP node records either entirely successful DNS name resolutions or entirely failed transactions, with no intermittent switching observed.

15. Key Findings

The distributed analysis of the infrastructure logs surfaced ten core insights:

1. **Extreme Record Cardinality:** The source IP attribute operates as a near-perfect primary key with 258,445 unique values across 258,445 total entries.

2. **High Resolution Failure Rate:** A striking 97.74% of all logging entries capture unresolved system exceptions, indicating severe misconfigurations or strict firewall drops across source subnets.
3. **Centralized Target Points:** The endpoint domain `int0.client.access.fanaptelecom.net` represents the most active single tracking target with 101 distinct access attempts.
4. **Structural Mutex Behavior:** Source nodes behave deterministically—completely segregating into successful resolvers or failed lookup emitters with no overlap.
5. **Structural Error Standardization:** Socket resolution exceptions are recorded uniformly using the standardized operating system string `[Errno 1] Unknown host`.
6. **Late-Binding Schema Viability:** Dynamic schema inference successfully prevented string truncated data corruption across un-escaped single quotes inside the list arrays.
7. **Typological Domain Dominance:** Classifying endpoint strings using regular expressions showed that 100% of successfully resolved records map to structured Domain Names rather than naked IP addresses.
8. **Granular Alias Spread:** The array explosion phase extracted 5,883 unique inverse pointers (such as `126.195.202.37.in-addr.arpa`), showing clear infrastructure distribution.
9. **Data Volume Reduction:** Programmatic cleaning reduced the active dataset size from 258,445 rows to 5,819 rows, highlighting how targeted filtering optimizes downstream model training.
10. **Zero Record Duplication:** A full cluster-wide signature match returned 0 duplicate records, confirming high data ingestion integrity.

16. Challenges Faced

During project execution, several data engineering and infrastructure bottlenecks were encountered and resolved:

- **Handling Stringified Python Collections:** The `alias_list` and `address_list` features were ingested as plain strings containing array punctuation (e.g., `"['item']"`). Using standard string splitting generated messy text remainders. This was resolved by wrapping Python's native `ast.literal_eval` function inside a type-safe Spark User Defined Function (UDF), enabling clean conversions into native `ArrayType` structures.
- **Managing Multi-Typology Null Definitions:** The dataset mixed empty cells (system nulls) with explicit string literals like `'NULL'`. Traditional missing-value filters skipped the text overrides. A multi-stage condition check was implemented using PySpark's `col().isNull()` combined with logical OR string matching to catch both types.
- **Google Colab Java Environment Dependencies:** Because PySpark requires an underlying Java Development Kit (JDK) runtime to drive JVM processes, initial execution calls threw environment errors. This was resolved by structuring automated environment setup steps inside the notebook to ensure appropriate dependencies are verified before starting the Spark context.

17. Learning Outcomes

This technical internship project advanced my engineering competencies across several areas:

- **Core Big Data Architecture:** Acquired practical experience with master-slave cluster scaling paradigms, distributed drivers, executors, and YARN resource management properties.
- **Advanced PySpark Manipulation:** Developed strong skills in building Spark SQL analytical pipelines, utilizing structural array transformations, data filtering, and high-performance multi-column sorting.
- **Data Quality Auditing:** Built capability in handling dirty data profiles by programming custom parsing scripts to resolve nested fields and multi-state null flags.
- **Analytical and Critical Thinking:** Learned to look past bulk data metrics to diagnose underlying network behaviors and assess the operational impact of systemic log errors.

18. Future Scope

The processing pipelines built in this internship can be expanded through several architectural enhancements:

- **Transition to Spark Streaming:** The batch ingestion setup can be extended into an active real-time data streaming pipeline by replacing `spark.read` blocks with `spark.readStream`, connecting directly to live network pipelines like Apache Kafka.
- **Integrating PySpark MLlib:** Network tracking records can be fed into Spark's native Machine Learning Library (MLlib) to train K-Means clustering algorithms, enabling automated, unsupervised anomaly and intrusion detection based on connection frequency profiles.
- **Cloud Databricks Deployment:** Scaling the pipeline requires migrating the notebook code into cloud-native distributed runtimes like Databricks or AWS EMR, using managed storage spaces like Amazon S3 or Azure ADLS for faster access performance.

19. Conclusion

The technical internship project **"Big Data Analytics Basics Using PySpark"** successfully addresses the design and deployment of an automated, scalable data processing architecture for high-volume network infrastructure logs. By exploiting the in-memory compute power of Apache Spark via its PySpark interface, this project successfully ingested, audited, parsed, and evaluated a highly sparse database of 258,445 access records.

The empirical results emphasize that while raw data streams often contain a high volume of noise and incomplete records—as shown by the 97.74% failed lookup rate—the systematic application of distributed transformations allows engineers to safely extract high-value analytics. The isolated 5,819 fully resolved logs and the extracted 5,883 domain aliases offer clear visibility into infrastructure mapping trends. Ultimately, this project proves that adopting framework abstractions like Spark DataFrames and custom UDF workflows

enables clean, scalable analysis of massive network infrastructure records, laying a solid foundation for robust enterprise data engineering systems.

20. References

1. **Apache Spark Core Documentation:** *Spark DataFrame Programming Guide and Internal Optimization Models.* Available at: https://spark.apache.org/docs/latest/sql-programming-guide.html
2. **PySpark Native API Manual:** *Structured Function References and UDF Implementation Paradigms.* Available at: https://spark.apache.org/docs/latest/api/python/index.html
3. **Apache Hadoop Core Architecture Guide:** *Distributed Filesystem (HDFS) and Block Replication Models.* Available at: https://hadoop.apache.org/docs/stable/
4. **Python Software Foundation Documentation:** *Abstract Syntax Tree (ast) Module and Literal Evaluation Security.* Available at: https://docs.python.org/3/library/ast.html